

Lista de Exercícios de Fixação

1. Escolha um projeto de software real ou acadêmico e crie uma estrutura de documentação usando o MKDocs. Organize a documentação de forma a cobrir instalação, configuração e uso de APIs.

Projeto escolhido: Agenda Farol, plataforma de agendamento comunitário desenvolvida para a Igreja Batista Farol de Joinville.

```
agenda-farol-docs/  
  mkdocs.yml  
  docs/  
    index.md  
    instalacao.md  
    configuracao/  
      ambiente.md  
      docker.md  
    api/  
      agendamentos.md  
      usuarios.md
```

Essa estrutura cobre instalação (requisitos e passo a passo), configuração (ambiente e Docker) e uso da API (endpoints de agendamentos e usuários), organizando o conhecimento do projeto de forma navegável.

2. Em um cenário colaborativo, divida um projeto de documentação entre diferentes membros de uma equipe. Cada membro deve ser responsável por uma parte específica, como instalação, API ou tutoriais. Use um sistema de controle de versão para coordenar o trabalho.

Aplicando ao projeto Agenda Farol, desenvolvido em equipe com Mateus e Jhonatan:

- **Patrick:** documentação de instalação e configuração do ambiente (Docker, variáveis de ambiente).
- **Mateus:** documentação dos endpoints da API de agendamentos.
- **Jhonatan:** tutoriais de uso para o perfil Admin (Anália) e para o Voluntário.

O controle de versão foi feito via Git, com o repositório hospedado no GitHub: cada integrante trabalhou em uma branch própria (ex.: docs/instalacao, docs/api) e abriu pull requests para revisão antes do merge no branch main, garantindo consistência no conteúdo final.

3. Crie um estudo de caso onde o MKDocs é usado para documentar um sistema completo. Inclua uma explicação sobre como a ferramenta foi integrada ao projeto e como o uso de documentação automatizada (com Swagger, por exemplo) facilitou o processo.

Sistema: API interna de agendamentos do Agenda Farol.

- **Desafio:** a equipe precisava de uma forma simples de documentar os endpoints (criar, listar e cancelar agendamentos) sem gerar trabalho manual repetido a cada alteração.
- **Solução:** o MKDocs foi integrado ao Swagger, com a especificação da API escrita em OpenAPI (openapi.yaml) e publicada dentro da pasta docs/. Isso permitiu gerar uma interface interativa, onde qualquer pessoa da equipe podia visualizar e testar os endpoints diretamente no navegador.

A documentação automatizada facilitou o processo porque, a cada mudança na API, bastava atualizar o arquivo openapi.yaml — sem reescrever manualmente páginas HTML — e o site já refletia a nova versão após o próximo mkdocs build.

4. Implemente um processo de revisão de documentação usando Git e pull requests. Cada membro da equipe deve fazer contribuições à documentação e outro membro deve revisar as mudanças antes de serem aceitas.

1. Cada membro cria uma branch a partir da main (ex.: git checkout -b docs/api-agendamentos).

2. Após editar os arquivos Markdown, o membro faz commit e push da branch para o GitHub.
3. Um Pull Request é aberto comparando a branch com a main.
4. Outro integrante da equipe revisa o conteúdo, sugere ajustes nos comentários do PR e só então aprova o merge.

Esse processo evita que informações incorretas ou mal formatadas cheguem à documentação publicada, já que passam por um segundo olhar antes de irem ao ar.

5. Crie uma documentação de API para um sistema fictício, incluindo exemplos de requisições e respostas. Use a especificação OpenAPI e integre o Swagger ao MKDocs para gerar uma interface interativa de documentação.

API fictícia: sistema de reserva de salas comunitárias.

```
openapi: 3.0.0
info:
  title: API de Reserva de Salas
  version: 1.0.0
paths:
  /salas:
    get:
      summary: Lista todas as salas disponíveis
      responses:
        '200':
          description: Lista retornada com sucesso
  /reservas:
    post:
      summary: Cria uma nova reserva
      responses:
        '201':
          description: Reserva criada com sucesso
```

Com esse arquivo em docs/api/openapi.yaml e o Swagger UI configurado no mkdocs.yml, a documentação gerada permite visualizar os exemplos de requisição e resposta e testar os endpoints /salas e /reservas diretamente pela interface interativa.